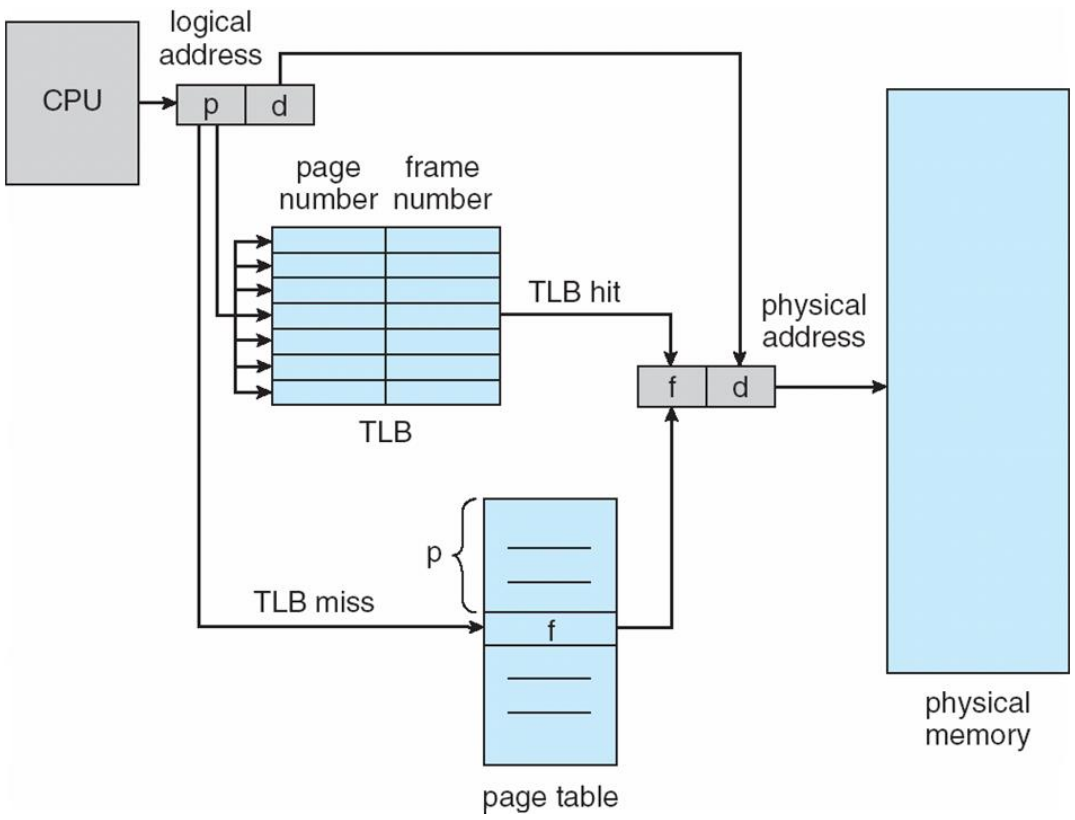
Cache-TLB Interactions

Sparsh Mittal

TLB = Translation Lookaside Buffer

Big picture


CPU
logical address
p
d
page number
frame number
TLB hit
TLB
f
d
physical address
p
TLB miss
f
page table
physical memory

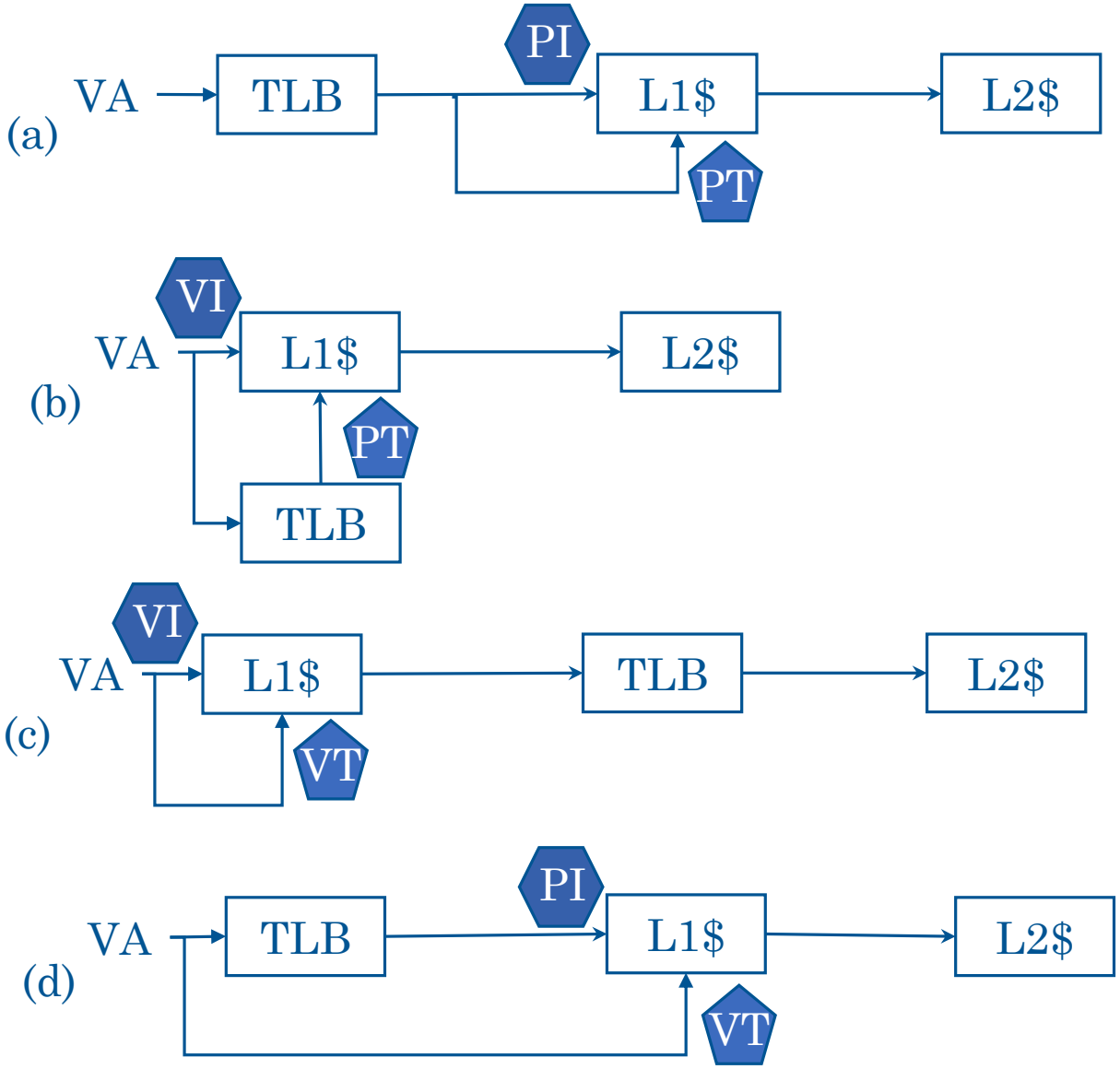
4 possible cache/TLB organizations

- Depending on whether index and tag of L1 cache is derived from virtual or physical address, we can have 4 possible organizations:
  - PI-PT cache (physically-indexed, physically-tagged)
  - VI-PT cache (virtually-indexed, physically-tagged)
  - VI-VT cache (virtually-indexed, virtually-tagged) (also called **virtual cache**)
  - PI-VT cache (physically-indexed, virtually-tagged)

4 possible cache/TLB organizations

Legend

xI = x (V/P) Index xT = x (V/P) Tag



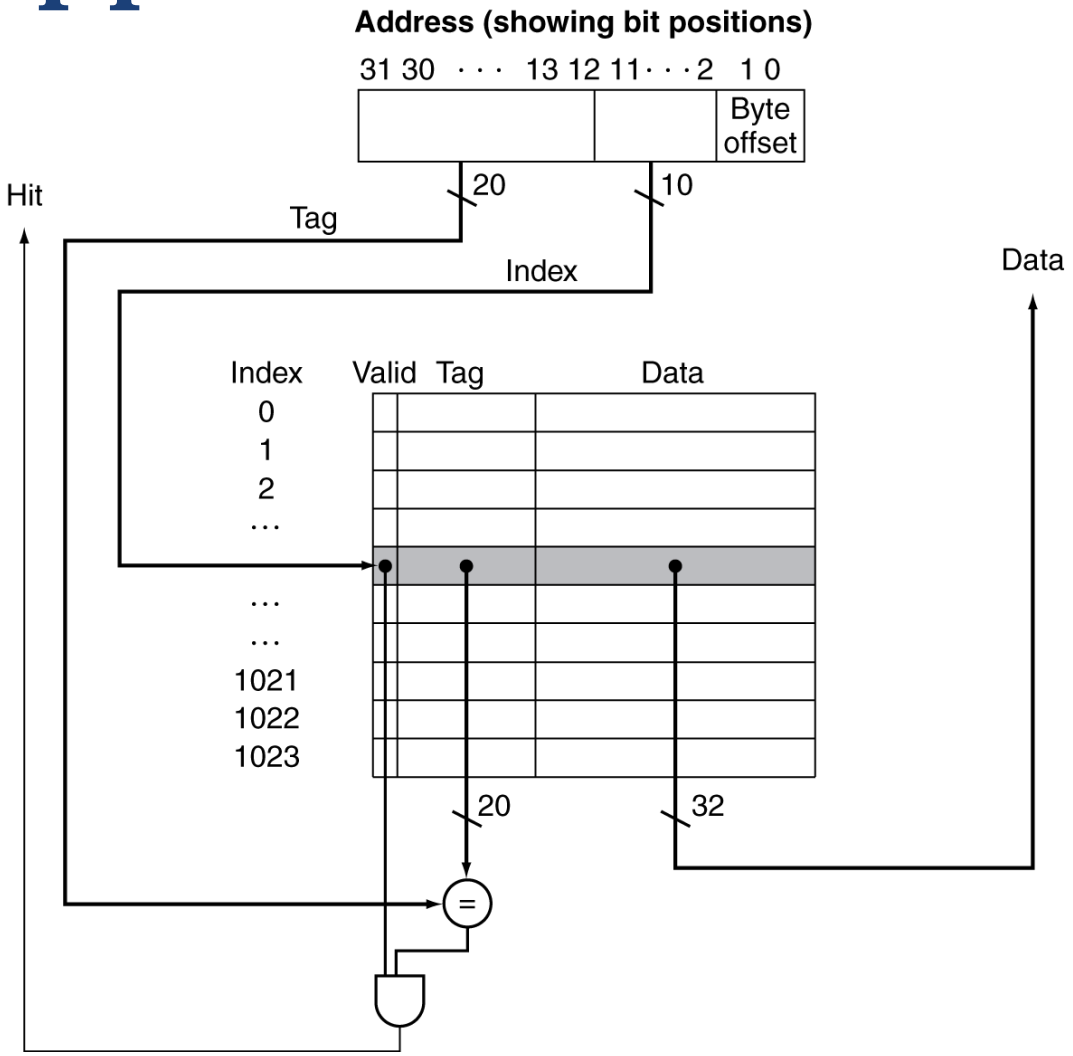
Physically indexed
Physically tagged

Virtually indexed
Physically tagged
Commonly used

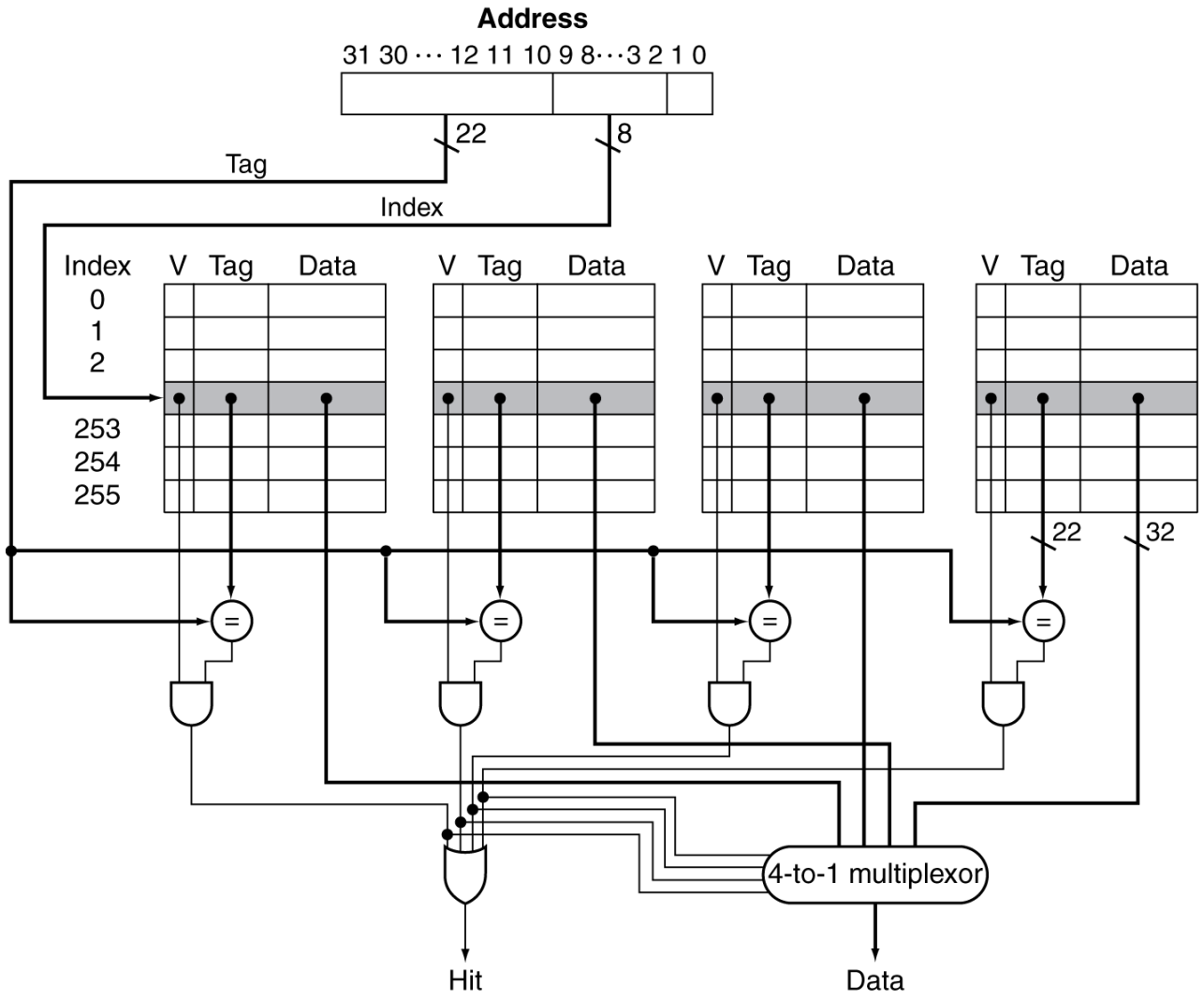
Virtually indexed
Virtually tagged
Reduces TLB access
But presents challenges

Physically indexed
Virtually tagged

Direct-mapped Cache



Set Associative Cache



Solved example

- Assume we get a total of 10,000 accesses to memory hierarchy. L1 cache miss-rate is 10%. Find number of accesses to TLB in each of 4 designs.
- Solution: Number of L1 misses is 1,000. VIVT design reduces TLB accesses.

Design	TLB Accesses
PIPT	10,000
VIPT	10,000
VIVT	1,000
PIVT	10,000

Comments on PIPT and PIVT

- Key observation: For accessing cache, we first need set-index and then need tag.
- PIPT: It does not reduce TLB accesses. It does not allow accessing TLB and cache in parallel.
- PIVT: It does not reduce TLB accesses. It does not allow accessing TLB and cache in parallel.
- For these reasons, above two designs are not used.

Comments on VIPT and VIVT

- VIPT: It does not reduce TLB accesses. However, it allows accessing TLB and cache in parallel.
- Hence, this design is commonly used.
- It suffers from synonym problem due to use of virtual-index. To solve this, we ensure cache sizes are such that all bits of set-index come from page-offset. [shown later in this PPT]

- VIVT: It reduces TLB accesses. No need of accessing TLB and cache in parallel.
- Hence, this design can be even better than VIPT.
- However, this design suffers from synonym and homonym problem.

Useful term: Homonyms

- **Homonyms:** Homonyms occur when two different processes have the same VA. These VAs are mapped to different physical locations.
- Challenge: If not disambiguated, incorrect data may be returned.

Page table of process 0

VPN	PFN
W	R
X	S

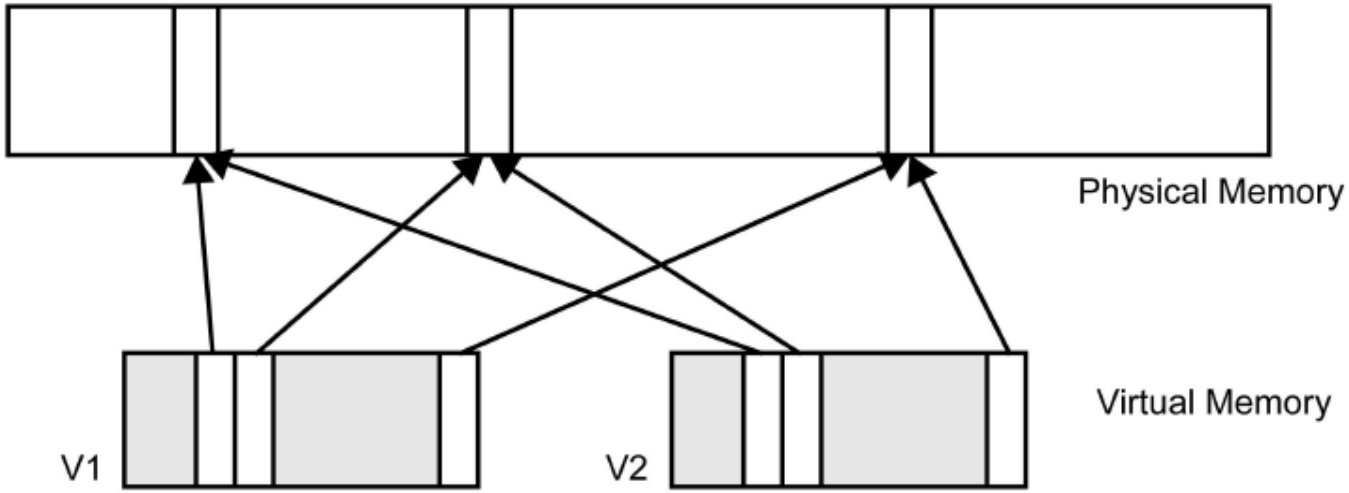
Page table of process 1

VPN	PFN
W	A
Y	B

- Solution: Use application-specific ID (ASID) to remove homonyms

Useful term: Synonyms

- **Synonyms:** Multiple different VAs mapping to same PA



- Challenge: These synonyms can reside in multiple sets in cache under different VAs. If one synonym of a block is modified, access to other synonyms with different VAs may return stale data
- Why synonyms occur? Shared libraries, shared data, copy-on-write pages within the same process, etc.

Qiu et al. TC'08

Solved question

- We have two processes sharing a processor.
- For process 0, VPN to PFN mappings are: [0, 1], [4, 7], [7, 2], [14, 3].
- For process 1, VPN to PFN mappings are: [0, 2], [5, 7], [8, 21], [13, 11].
- If the page size is 8B and we use VIPT design, then find a physical address whose data can exist in two different locations in the cache.
- To find any one physical address, we can set the page offset to be any value between 000 to 111. Let us discuss the solution with 000.
- Any byte in frame 7 is seen as 2 different virtual pages from processes 0, 1. Physical address of the first byte in frame 7 = 111000.
- Similarly, any byte in frame 2 is seen as two different virtual pages from processes 0, 1. Physical address of the first byte in frame 2 = 010000. This is also correct answer.

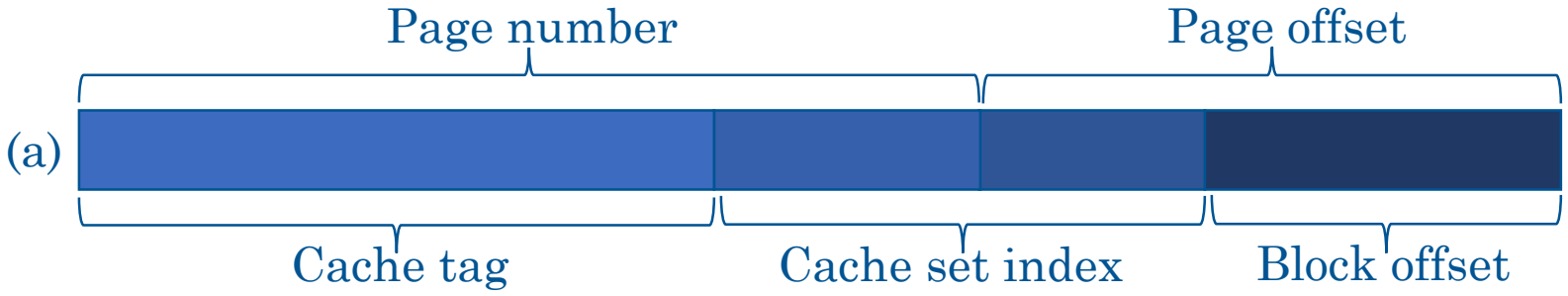
(VPN= virtual page number, PFN = physical frame number).

Solution for synonym problem

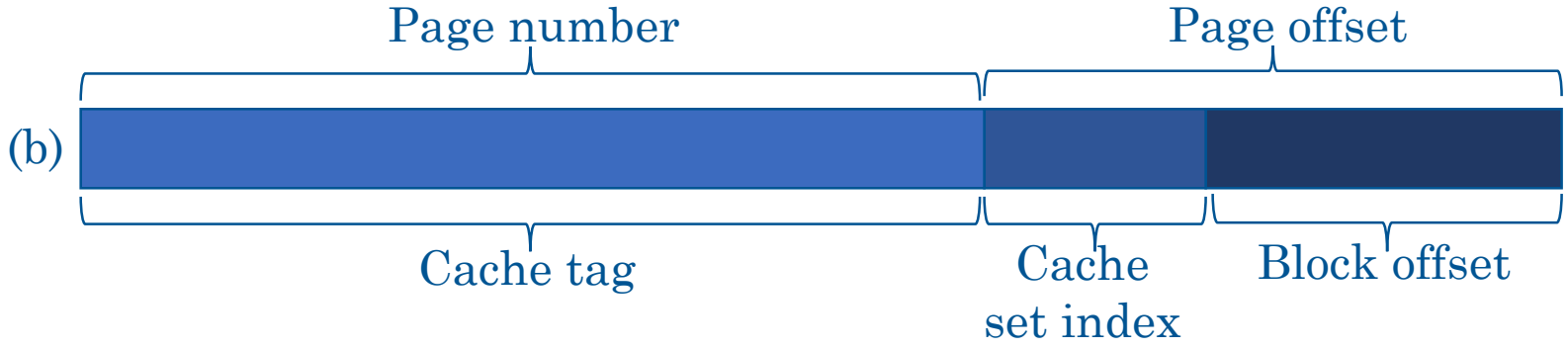
- If all bits of cache set come from page offset
- => Cache set location of an address is independent of VA-to-PA translation
- Then, synonym problem is avoided.
- This is shown on the next slide.

Physical address subdivision

Address subdivision in **general case**



Special case: all bits of cache set come from page offset
=> Cache set location of an address is independent of VA-to-PA translation



TLB access not required for obtaining index

Solved question

- A processor uses a page size of 16 KB. L1 cache must be 4-way set-associative and has a block size of 64B.
- You are implementing a virtually indexed physically tagged L1 cache, such that the TLB access is not required for indexing into the L1 cache. What is the largest L1 cache that you can design?
- Solution:
- Maximum number of sets in L1 is equal to (16*1024)/64 = 256
- Number of ways = 4
- So, largest cache size = 256 * 4* 64B = 64KB
- If cache is larger than this, some bits of set-index will come from page-number, so TLB access will be required.

References

- Sparsh Mittal, “A Survey of Techniques for Architecting TLBs”, Concurrency and Computation, 2017 (link)